Toward a Control Architecture for Composable AI Execution Systems

An Empirical Survey of Architectural Strengths and Anti-Patterns in Six Production AI Frameworks, with a Theoretical Model for Control Discipline Layer Specification

David E. King · ActionPace.com · third-place.space/ai

AI as Creative Partner Research Series · Working Paper · April 2026
arXiv forthcoming · cs.SE (primary) · cs.AI (secondary)

ABSTRACT

Six production AI execution frameworks — AutoGen, CrewAI, LangGraph, n8n, OpenClaw, and PydanticAI — are surveyed using ensemble divergence methodology applied with a strength-extraction framing. A three-phase funnel (map, surgical, synthesis) covering 1,269 files at a total analysis cost of $6.58 identifies eight distinct architectural strengths and four structural anti-patterns across the surveyed systems. The strengths represent genuine solutions to specific architectural problems: provider normalization, context threading, type-driven routing, transport abstraction, state management, and session compaction. The anti-patterns represent a consistent class of orphaned invariants — contracts that execution components depend on but that no single component owns or declares. The survey finds no framework that has integrated control discipline at the architectural level. A theoretical model specifying a control discipline layer is developed from the survey findings. The model comprises four commitments — session boundary enforcement, integrity monitoring, composition seam governance, and attention preservation — specified as interface-level contracts. The model addresses the structural origin of the productive-but-tired paradox: the systematic transfer of implicit contract burdens onto the human operator as execution capability grows.

1 · INTRODUCTION

The Shadow of Execution

At first it feels like leverage. Tasks disappear. Things that previously consumed hours resolve in minutes. The morning looks different — lighter, faster, more spacious. But after some weeks, something else arrives. The work has not gone. It has moved. Into the gaps between the systems. Into the monitoring, the correction, the prompt refinement, the verification of outputs that looked right until they were acted on. They are doing more, not less. More tired, not less. The system that was supposed to free them has become the thing they serve.

This is the shadow of execution.

The enthusiasm with which these systems were adopted was not irrational. Execution power is genuinely valuable. The ability to delegate repetitive tasks, parallelize information retrieval, normalize responses across incompatible provider formats, and maintain computational state across sessions — these are real capabilities that address real frictions in knowledge work. Practitioners who built the first multi-agent setups were not confused about the value proposition. They were correct that the tools worked and that the tools were accessible. What many discovered only after building is that the systems, in working, had created a new class of problem that the systems themselves were not designed to address. Diagram 1

> *The central thesis of this paper is not that execution power is problematic. It is that execution power without a corresponding control discipline systematically transfers complexity onto the human operator — and that this transfer is not incidental but structural, embedded in the same architectural assumptions that made each framework effective at what it does well.*

The problem presents as a spectrum. At one end, a practitioner adds a single agent to handle email triage and discovers that the agent occasionally misclassifies messages in ways that require more attention to catch than the original task required to perform. At the other end, a practitioner running six specialized agents across their entire workflow finds, paradoxically, that their working hours have lengthened and their cognitive load has increased.

This pattern — more execution, more fatigue, more complexity transferred to the human boundary — has been documented across regional adoption studies, practitioner accounts, and engineering post-mortems. It is not a bug in specific systems. It is a property of a class of systems in which execution capability has been optimized without a corresponding investment in what we term control discipline: the architectural commitment to governing the boundaries, costs, and human interaction surface of execution power.

1.1 · THE STRUCTURAL QUESTION

What Frameworks Solve and What They Do Not

A careful structural reading of existing execution frameworks reveals that each has solved a specific and genuine problem. These are not approximate solutions or premature optimizations. They are, in several cases, sophisticated engineering achievements that address real architectural difficulties. The normalization of incompatible provider streaming semantics. The threading of runtime services through stateful graph execution without polluting every function signature. The derivation of message routing decisions from static type declarations. The management of concurrent session state under compaction pressure. Each of these represents a genuine contribution to the architecture of AI-augmented systems.

What the structural survey finds, repeatedly and across frameworks with very different design philosophies, is a consistent gap between what each framework solves and what it leaves undeclared. The gap takes different forms — implicit initialization ordering, ambient context dependencies, provider name parsing conventions, constructor side effects — but its structural signature is consistent: contracts that multiple components depend on but that no single component owns. These orphaned invariants are not bugs. They are the price of architectural commitments that were made without sufficient attention to what the calling system would need to know in order to compose safely with the execution layer. Diagram 2

More significantly, this gap does not close as execution capability grows. It widens. The more sophisticated the execution layer, the more implicit the contracts it places on its environment. The more capable the framework, the more the human operator must understand without being told. This is the structural origin of the fatigue pattern: practitioners are not failing to use the tools correctly. They are paying the accumulating cost of implicit contracts that the tools do not declare and cannot self-govern.

1.2 · THE CONTROL DISCIPLINE GAP

What Has Not Been Built

The survey finds no framework that has integrated control discipline at the architectural level. Control discipline, as we define it here, comprises four commitments absent from every surveyed framework. The first is *session boundary enforcement*: the architectural guarantee that execution operates within bounded contexts rather than accumulating unboundedly across sessions. The second is *integrity monitoring*: the continuous, lightweight evaluation of execution outputs against declared invariants — not as a post-hoc quality check but as a live governance layer running alongside generation. The third is *composition seam governance*: the explicit management of the boundaries at which different execution components interact, such that the implicit contracts between them are surfaced, owned, and testable rather than silently assumed. The fourth is *attention preservation*: the architectural commitment to treating human attention as the scarce resource the system must serve rather than the resource the system may consume without limit.

> *These four commitments do not require a new execution framework. They require a layer of a different kind — one that sits above execution, selectively composes existing execution strengths, and governs the boundaries that execution layers are structurally unable to govern from within the same architectural assumptions that made them effective.*

1.3 · STRUCTURE OF THIS PAPER

What Follows

Section 2 describes the survey methodology. Section 3 presents the pattern catalog: eight named architectural strengths and four named anti-patterns. Section 4 develops the theoretical model of the ideal composition architecture. Section 5 specifies the four control discipline commitments as interface-level contracts. Section 6 discusses implications for practitioners and framework developers. Section 7 states limitations and identifies directions for further empirical work.

Throughout, we maintain the distinction between what the empirical survey establishes — patterns observed in specific systems at specific confidence levels — and what the theoretical model proposes — a composition architecture that has not yet been implemented and whose claimed properties are therefore theoretical rather than empirically confirmed. The paper's contribution is the theory, grounded in the empirical findings, not the implementation.

1.4 · RELATIONSHIP TO EXISTING WORK

Formal Context and Complementary Contributions

The structural gap this paper identifies has received formal theoretical treatment in parallel work. Fernandez (2026) proves the Non-Identifiability Theorem: that the admissible behavior space Ω established at agent admission time is not in the σ-algebra generated by the local enforcement signal E. In practical terms, any enforcement system that evaluates actions point-wise against a rule set is structurally blind to distributional drift — an agent can depart systematically from its admission-time behavioral contract while satisfying every local check. Fernandez proposes an Invariant Measurement Layer (IML) as the detection mechanism, using ensemble methods to restore observability in the region where enforcement is blind.

The present paper is complementary rather than duplicative. Where Fernandez establishes the mathematical necessity of a higher-order governance layer, this paper provides the empirical evidence of where that necessity manifests in production systems. Three distinctions are worth stating precisely.

First, the method differs. The Fernandez framework operates on behavioral traces at runtime. The present survey operates on source code at the structural level — identifying the architectural commitments that make invariant orphaning inevitable before any agent has run. The survey does not observe drift; it identifies the seams where drift will occur.

Second, the object of study differs. Fernandez addresses behavioral deviation from admission-time contracts in running systems. This survey addresses the structural conditions in production codebases that produce orphaned invariants as a property of the framework's architecture, independent of any specific runtime behavior.

Third, the contribution differs. The Fernandez IML answers: how do we detect when drift has occurred? The composition seam governance specification in Section 5 answers a prior question: how do we ensure that the implicit contracts are named, owned, and testable before execution begins, so that what the IML must detect has been reduced in scope?

OpenAI's April 2026 Codex changelog introduced multi-agent relationship tracing as an observability layer — confirming that session boundary visibility is recognized as a production requirement, while remaining a post-hoc diagnostic rather than a pre-execution governance mechanism. This distinction is precisely what the control discipline layer specification addresses.

2 · METHODOLOGY

Ensemble Divergence and the Strength-Extraction Framing

The empirical work underlying this paper used ensemble divergence methodology applied with a strength-extraction framing — a deliberate departure from the vulnerability-finding orientation in which the methodology was originally developed. This section describes both the methodology and the framing shift, and explains why the shift was necessary to produce findings useful for a theoretical composition model rather than a fault-finding report.

2.1 The Three-Phase Funnel

The survey operated in three sequential phases, each filtering the scope of the next. Diagram 3

The *map phase* applied a single lightweight model to every file in each codebase's target directory, scoring each file on a five-dimension opacity rubric: undeclared preconditions, global state risk, implicit coupling, contract completeness, and structural centrality. Files above the threshold on both opacity and activity churn were classified as fire traps; high opacity with low churn as frozen complexity; low opacity with high churn as active healthy; low opacity and low churn as stable. The map phase covered 1,269 files across six codebases at a total cost of $5.91 — approximately $0.005 per file.

The *surgical phase* applied a three-model ensemble to high-priority files identified by the map phase — predominantly frozen complexity targets whose high opacity and low churn indicated crystallized architectural commitments. The three models represented architecturally different systems, ensuring findings confirmed across all three were robust to model-specific training biases. Total surgical phase cost: $0.58, covering approximately 60 files — fewer than 5% of scanned files.

The *synthesis phase* applied a single higher-capability model to the surgical results for each framework, clustering findings into named patterns, resolving naming divergence, and classifying each cluster as strength, debt, or ambiguous. Total synthesis cost: $0.09 across seven synthesis runs. Diagram 4

2.2 Divergence as Signal

In ensemble divergence methodology, model disagreement is not noise — it is the primary analytical signal. When all three models independently identify the same contract in a file, often using different names, convergence is treated as high-confidence confirmation. The number of independent confirmations becomes the reliability measure: a contract confirmed at 9× has been named by three models analyzing three different files, each time independently.

This represents a deliberate epistemological departure from consensus-based ensemble methods. Existing approaches treat agreement as confidence. This methodology inverts that assumption: disagreement is the primary signal, identifying regions of structural ambiguity and competing architectural interpretations that consensus-based methods suppress. Where models agree, confidence is established. Where they disagree, a structurally significant boundary has been located. The composition seams that carry the highest invariant-orphaning risk are precisely the regions where independent models — given the same code — diverge in their characterization of what obligation is being created.

2.3 The Strength-Extraction Framing

Previous applications of this methodology targeted vulnerability finding. For this survey the question changed: not where is this code dangerous, but what is this code actually solving that its documentation does not surface. The strength-extraction prompt asked models to determine what each file handles so that callers do not have to, and to classify each implicit contract as strength, debt, or ambiguous. The addition of the nature classification field is what made the comparative strength-debt profiles in Section 3 possible.

2.4 Calibration

Each map phase run was calibrated against a known-load-bearing file — the file predicted on independent grounds to rank highest in structural complexity for each codebase. In every codebase surveyed, the calibration target ranked in the top five files by opacity score without any prompt engineering or hints. This result was consistent across six architecturally different codebases in three programming languages (Python, TypeScript, and mixed), providing confidence that the opacity scoring is measuring genuine structural properties rather than superficial textual features.

3 · PATTERN CATALOG

Architectural Strengths and Anti-Patterns Across Six Systems

The survey produced 68 named contract clusters across six frameworks. After cross-framework deduplication, eight distinct architectural strengths and four anti-patterns emerged as the most robustly confirmed findings. Diagram 5 Diagram 6

Confirmation counts reflect the number of independent model-file observations that named the same contract. A count of 9× indicates the highest confidence in the study; a count of 2× indicates a finding present but not yet robustly confirmed.

3.1 Architectural Strengths

Provider Normalization Boundary

9× confirmed strength

A single factory boundary that absorbs all provider-specific message formats, tool definition schemas, and response structures, exposing a unified typed protocol to callers. Provider-specific quirks — streaming semantics, content type handling, tool calling conventions — are resolved inside the boundary and invisible outside it.

Architectural significance: Its presence removes an entire class of caller complexity. Systems building on this capability gain multi-provider portability without implementing normalization themselves.

Layer: Message protocol · Composition seam: factory function · Source: PydanticAI

Context Threading Pattern

9× confirmed strength

A mechanism for propagating typed runtime context — configuration, services, dependencies — through every layer of agent execution without requiring each layer to declare the full context as a parameter. The context is injected at the system boundary and threaded automatically.

Architectural significance: Eliminates one of the most common sources of function signature pollution in agent systems. Components can access what they need without the calling chain carrying it explicitly.

Layer: Routing · Composition seam: context accessor · Source: LangGraph, PydanticAI

Type-Hint Routing Bridge

6× confirmed strength

Message routing decisions are derived from static type annotations on handler functions rather than from explicit routing configuration. The router inspects the type signature and dispatches automatically; the developer declares what a function accepts and the system routes accordingly.

Architectural significance: Routing logic becomes a property of the code structure rather than a separate configuration artifact. Types serve double duty as both documentation and dispatch logic.

Layer: Routing · Composition seam: type-driven dispatch · Source: AutoGen

Interceptor Transport Weaving

5× confirmed · strength

A hook-based mechanism for inserting cross-cutting concerns — logging, retry, rate limiting, observability — into the transport layer without modifying the handlers that use the transport. Interceptors are registered at the transport boundary and applied automatically to all traffic passing through it.

Architectural significance: The governance insertion point for a control discipline layer. The interceptor hook is the cleanest composition seam in the transport layer — a governed boundary where caller obligations are declared and side effects are visible.

Layer: Transport · Composition seam: interceptor registration API · Source: n8n, CrewAI

Structured Output Protocol

5× confirmed · strength

A standard protocol for requesting and validating structured outputs from language models, using schema declarations that are portable across providers. The protocol handles schema translation, validation, and retry logic for malformed outputs, exposing a typed result to callers.

Layer: Transport · Source: PydanticAI, LangGraph

Three-Tier Precedence Stack

3× confirmed · strength

A configuration system with three levels of precedence — default, session, and invocation — where invocation-level settings override session-level, which override defaults. The precedence logic is centralized and declared; callers can override at any level without knowledge of lower levels.

Layer: State / Persistence · Source: PydanticAI

Session Compaction Lifecycle ★

3× confirmed · unique — no equivalent in any other surveyed system

An explicit lifecycle for managing long-running sessions: a bounded context window with a defined compaction trigger, a compaction strategy that preserves summary state while releasing detailed history, and a clean continuation path from the compacted state. The lifecycle is declared and owned; the caller does not need to manage the session window directly.

Architectural significance: The only pattern in the catalog that directly addresses the long-running session management problem. Its existence in one framework is evidence that the problem is solvable; its absence in all others is evidence that it has not been recognized as a framework responsibility.

Layer: State / Persistence · Source: OpenClaw

Finish Reason Normalization

2× confirmed · strength

Normalization of provider-specific finish reason codes into a unified enumeration. Callers can branch on finish reason without handling provider-specific strings.

Layer: Message protocol · Source: PydanticAI

3.2 Structural Anti-Patterns

The four anti-patterns below were identified across multiple frameworks at varying confidence levels. Each represents a structural commitment that produces orphaned invariants — contracts that the framework depends on but that no component declares ownership of.

Lazy Initialization Trap — 9× confirmed

Agent or tool objects require a separate initialization call after construction before they can be used correctly. The construction call succeeds regardless of whether the subsequent initialization will succeed. Credentials are validated, connections are established, or resource acquisition occurs at first invocation rather than at construction, with no explicit notification to the caller.

Structural consequence: Initialization failures arrive as runtime errors during task execution rather than as construction-time failures. Callers cannot distinguish between "object created successfully" and "object will succeed when invoked." The trap is the highest-confirmed finding in the anti-pattern catalog.

Wrapper strategy: An explicit initialization method that validates credentials and returns a result before any execution begins, converting runtime discovery of initialization failure into construction-time failure.

Ambient Context Dependency — 3–6× confirmed

Agent instantiation or execution requires ambient context variables to be set in the calling environment before the call, without this requirement being declared in any interface signature, type annotation, or documentation. The dependency is real and non-optional; its absence produces silent failure or incorrect behavior rather than a clean error.

Structural consequence: Testing and external composition become unreliable because the ambient context is set correctly in the framework's own test harness and incorrectly everywhere else. This is the canonical form of orphaned invariant — a contract that the system depends on but that no single component declares ownership of.

Wrapper strategy: Make instantiation context an explicit constructor parameter, converting ambient discovery into declared dependency.

Provider Name Parsing Burden — 3× confirmed

Provider identification relies on string parsing conventions — a format like "provider:model" — rather than on structured specification objects. The parsing logic and its edge cases are embedded in the factory function without documentation. Callers must understand naming conventions that are implicit in the implementation rather than declared in any interface.

Wrapper strategy: A structured provider specification object that makes the provider and model explicit as typed fields, with string parsing as a legacy compatibility path rather than the primary interface.

Constructor Side Effects — 3× confirmed

Object construction triggers side effects — global state mutation, lifecycle hook registration, dependency injection container modification — that affect the surrounding environment without declaration. The object appears to be a value being created; it is actually an operation being performed.

Structural consequence: Composition becomes order-dependent in ways not visible from the interface. The side effects accumulate across a session without any mechanism for reversing them.

Wrapper strategy: A static factory method that returns an initialized instance, with all side effects explicit in the factory's interface, and a corresponding teardown method that cleanly reverses them.

4 · THEORETICAL MODEL

The Control Discipline Layer

The pattern catalog establishes that existing execution frameworks have solved distinct and genuine problems at specific architectural layers. No single framework has solved all of them. No framework has addressed the control discipline problem. This section develops the theoretical model of a composition architecture that takes the catalog's findings as input: a system that composes the genuine strengths of the execution layer while adding the governance that the execution layer cannot provide from within its own commitments. Diagram 7

4.1 The Composition Principle

The starting insight is that the eight architectural strengths are not competing capabilities. They address different problems at different layers of the system. A composition architecture can use each strength for the problem it solves while delegating nothing to any framework that it has not chosen to delegate.

This is what distinguishes the control discipline layer from a meta-framework. A meta-framework wraps existing frameworks and inherits their implicit contracts along with their capabilities. A control discipline layer selects specific capabilities from specific frameworks at specific boundaries — the provider normalization boundary from one, the transport weaving hook from another, the three-tier precedence logic from a third — and composes them under a unified governance contract. The governance contract is the layer's own contribution. The capabilities are borrowed but governed.

The practical consequence: when a capability is composed into the control discipline layer, its associated anti-patterns are not inherited automatically. The lazy initialization trap is a property of the framework's own object lifecycle — but a control discipline layer that invokes the capability through an explicit initialization boundary can prevent the trap from propagating to its own callers.

4.2 The Constitutional Principle

The control discipline layer governs through structure, not through surveillance. Rules are established before execution begins, not applied as interruptions during it. This is the constitutional principle: the system's constraints are declared at the boundaries that precede action, not enforced by reviewing actions after they occur.

The practical distinction matters for attention preservation. A surveillance model requires continuous human attention to the review process. A constitutional model requires human attention at the declaration points but not at every subsequent execution cycle. The practitioner sets the rules; the system enforces them; the practitioner sees results that have already passed through the enforcement layer.

4.3 The Attention Scarcity Principle

The deepest architectural commitment in the control discipline layer is the recognition that human attention is the system's most scarce and most valuable resource — and that every architectural decision must be evaluated against whether it conserves or consumes attention.

Existing execution frameworks optimize for execution capability, treating attention as available in whatever quantity the practitioner brings. The implicit model is that the practitioner will read outputs, evaluate results, correct errors, and coordinate components as needed. This works when execution capability is modest. It fails as execution capability grows, because oversight requirements grow with the execution surface area while the practitioner's attention remains constant.

The control discipline layer inverts this. Every capability included in the system must justify its attention cost. Every output the system produces must either pass through integrity monitoring before reaching the practitioner or be ignorable without cost. The practitioner should be able to not read most of what the system produces and lose nothing of value — because value has been separated from volume before it arrives.

5 · INTERFACE SPECIFICATIONS

What Each Commitment Must Guarantee

This section specifies the four control discipline commitments as interface-level contracts — properties that any compliant implementation must exhibit, stated without reference to specific implementation approaches.

5.1 Session Boundary Enforcement

> **What this commitment must guarantee**
>
> Every execution operation occurs within a bounded session. A session is a unit of execution with a declared scope, declared constraints, and a guaranteed cleanup path. No execution state persists across session boundaries unless explicitly promoted by the practitioner. No ambient state accumulated in one session is visible in the next.
>
> The session boundary is enforced before execution begins, not after. Entry to a session requires declaration of intent and constraints. Exit from a session guarantees cleanup regardless of whether execution succeeded, failed, or timed out. Between sessions, the system holds no running processes. The practitioner is not obligated to monitor what is not running.

What this does not mean: Session boundary enforcement does not prevent long-running operations. It requires that long-running operations be explicitly declared as such, with checkpointing and recovery paths. The session compaction lifecycle pattern is precisely the implementation of long-running session management that the specification permits. Diagram 8

5.2 Integrity Monitoring

> **What this commitment must guarantee**
>
> Every significant output produced during a session is evaluated against a set of declared monitors before reaching the practitioner. Monitors are lightweight relative to the primary generation model. They run in parallel with generation, not sequentially after it. At minimum, four monitor types must be supported: an intent monitor; a liberty monitor; a coherence monitor; and a value alignment monitor. Output that passes all monitors is delivered; output that fails is either revised or discarded, with the failure logged but not presented as output the practitioner must evaluate.

What this does not mean: Integrity monitoring does not guarantee output quality in an absolute sense. It guarantees that output reaching the practitioner has passed structural integrity checks against declared invariants. The quality of monitoring is proportional to the quality of the declarations the practitioner provides. Diagram 9

5.3 Composition Seam Governance

> **What this commitment must guarantee**
>
> Every boundary at which two execution components interact is explicitly governed: the contract between components is named, declared, owned by the governance layer, and testable in isolation. No implicit contract between components is permitted to remain implicit. The governance layer becomes the owner of inter-component contracts that previously had no owner. It does not invent new contracts; it surfaces, owns, and makes testable the contracts that already existed implicitly.

What this does not mean: Composition seam governance does not require formal verification. It requires that contracts be declared and testable. Diagram 10

5.4 Attention Preservation

> **What this commitment must guarantee**
>
> The system is designed so that most of what it produces can be ignored by the practitioner without cost. The system does not require continuous monitoring. The filtering of volume from signal occurs inside the system before outputs reach the practitioner. Together, the four commitments define a system in which the practitioner's role is direction and selection — not surveillance and correction. The practitioner looks for substance. The system handles volume.

What this does not mean: Attention preservation does not remove the practitioner from the system. It changes what the practitioner's attention is directed toward. The practitioner remains the director of the system's goals and constraints.

6 · DISCUSSION

Implications and the Broader Question

The survey findings arrive in the context of a rapidly developing formal literature on the governance of autonomous systems. The Non-Identifiability Theorem (Fernandez, 2026) has established that local enforcement cannot guarantee adherence to admission-time behavioral contracts. ACP (2026) has introduced admission control for agent actions. CAAF has formalized the agent harness as a versioned enterprise asset. COCO has operationalized heterogeneous cross-validation for multi-agent workflow reliability. What this survey adds is the empirical surface: where the failure manifests in the six production frameworks practitioners are actually building on today.

6.1 For Practitioners

The pattern catalog offers a framework for evaluating existing execution systems independent of marketing claims or community reputation. Eight capabilities have been confirmed at high confidence; they are real and worth composing with. Four structural patterns consistently produce hidden burdens; they are worth wrapping at the boundary rather than inheriting directly. The theoretical model also offers a diagnostic for the productive-but-tired pattern. The prescription is not to dismantle what has been built but to add the four commitments as a layer above what exists.

6.2 For Framework Developers

The anti-pattern catalog is offered constructively. The lazy initialization trap, the ambient context dependency, the provider name parsing burden, and the constructor side effects are all addressable at the framework level. The session compaction lifecycle is offered as a model: its existence in one framework is evidence the problem is solvable; its absence in every other framework is evidence the problem has not been recognized as a framework responsibility.

6.3 The Broader Question

The patterns found — genuine execution strengths, consistent control discipline gaps — reflect a moment in which execution power has grown faster than the discipline to govern it. The AI transition is producing the same structure as prior technology transitions at different speed and scale. The human cost is being paid in attention, cognitive load, and the productive-but-tired paradox. The question the theoretical model tries to answer is whether the governance capability necessary to make AI-augmented work sustainable can be built and specified with sufficient clarity that it can be implemented, evaluated, and improved. The answer this paper offers is yes, with conditions.

7 · LIMITATIONS AND FUTURE WORK

What This Paper Does Not Establish

7.1 Scope

The survey covers six frameworks — sufficient to identify recurring patterns but not to establish universality. Three are Python codebases; two are TypeScript; one is mixed. The TypeScript findings should be treated as a methodological extrapolation until a dedicated TypeScript validation run is completed.

7.2 Ceiling Compression

The map phase uses a 300-line corpus slice per file. Five files in the OpenClaw survey scored exactly 0.880 — the effective ceiling — indicating ceiling compression prevented differentiation among the most complex files. Findings about the largest files should be treated as lower-confidence.

7.3 The Labeling Artifact

The AutoGen synthesis result contains a labeling artifact in which the `real_architectural_problem` field was populated with the framework name from the previous synthesis run. The content is correct; the field label is not. This does not affect substantive findings.

7.4 The Theory Has Not Been Implemented

The theoretical model in Sections 4 and 5 specifies a composition architecture that has not been built. Its properties are architectural claims, not empirically confirmed capabilities. Implementation and evaluation are the work of subsequent papers.

7.5 Future Work

Three directions: the overlapping slice approach for large files; expanding the survey to additional frameworks; and building a prototype system to the Section 5 specification and evaluating whether it exhibits the specified properties.

CONCLUSION

The Work That Remains

The survey finds eight genuine architectural strengths confirmed at high confidence across six production AI execution systems, and four structural anti-patterns confirmed with equal rigor. Every strength represents a problem that has been solved — normalization, threading, routing, transport abstraction, state management — and that a composed architecture can borrow rather than rebuild. Every anti-pattern represents a hidden contract that a governed composition boundary can prevent from propagating to the practitioner.

What no surveyed system provides as an integrated compositional layer is the control discipline specification: the session boundary enforcement that prevents unbounded accumulation, the integrity monitoring that filters outputs before they reach human attention, the composition seam governance that gives owned homes to the orphaned invariants that cause the longest-surviving failures, and the attention preservation commitment that treats human attention as the system's most scarce resource rather than its most available one.

The theoretical model is coherent. The specifications are actionable. The empirical foundation is sufficient to ground the claims without overclaiming what the survey alone can establish. The work that remains is implementation — building the thing, evaluating whether it does what the theory predicts, and reporting back. That is the subject of the work that follows this paper.

SELECTED REFERENCES

Fernandez (2026). *From Admission to Invariants: Measuring Deviation in Delegated Agent Systems.* arXiv:2604.17517.

Agent Control Protocol ACP (2026). *Admission Control for Agent Actions: Technical Specification and Reference Implementation.* arXiv:2603.18829.

CAAF (2026). *Harness as an Asset: Enforcing Determinism via the Convergent AI Agent Framework.* arXiv:2604.17025.

COCO (2026). *Cognitive Operating System with Continuous Oversight for Multi-Agent Workflow Reliability.* arXiv:2508.13815.

Sonar (2026). *State of Code Developer Survey Report 2026.* Sonar Source. 1,149 respondents, October 2025.

Leveson, N. (2012). *Engineering a Safer World: Systems Thinking Applied to Safety.* MIT Press.

Meyer, B. (1992). *Applying Design by Contract.* IEEE Computer, 25(10), 40–51.

AI USE DISCLOSURE

DAISY — DISCLOSURE OF AI USE IN RESEARCH

This paper was produced through a human-directed AI collaboration methodology in which AI language models served as analytical instruments under sustained human intellectual oversight. The research questions, theoretical framing, pattern classification criteria, and architectural specifications were originated, directed, and validated by the human author. AI models (Claude Sonnet, Claude Haiku, and Gemini 2.5 Flash Lite) were used as ensemble survey instruments, with ensemble divergence between models serving as the primary structural signal. The ensemble divergence methodology, the calibration target design, the differential analysis technique, and the interpretation of convergent and divergent findings were human-designed and human-validated throughout. All empirical claims — confirmation counts, opacity scores, cost figures, and pattern classifications — were reviewed by the human author and confirmed against raw model outputs. The theoretical model in Sections 4 and 5 represents original human synthesis grounded in the survey findings, not model-generated speculation.

David E. King · ActionPace.com · third-place.space/ai

AI as Creative Partner Research Series · Working Paper · April 2026 · Draft v0.2

APPENDIX A · WORKED EXAMPLE

Ensemble Divergence in Practice: LangGraph pregel/main.py

The following documents a four-model ensemble run against `libs/langgraph/langgraph/pregel/main.py` (3,773 lines), illustrating the convergence/divergence dynamic described in Section 2.2 with a concrete example from a public open-source codebase. The ensemble pairs two Apertus models (Swiss AI, Llama-family, CSCS) with two Mistral models (French, independent architecture) — a deliberately heterogeneous combination that tests whether structural signals are robust across training lineages, not just model sizes.

A.1 Overall Opacity Scores

MODEL	SCORE	CLASSIFICATION	TRAINING LINEAGE
Apertus-8B-Instruct-2509 (Q5_K_M)	78/100	HIGH	Swiss / Llama-family
Mistral-7B-Instruct-v0.3 (Q5_K_M)	60/100	HIGH	French / Mistral arch.
Apertus-70B-Instruct-2509 (Q5_K_S)	61/100	HIGH	Swiss / Llama-family
Mistral-Nemo-Instruct-2407 (Q5_K_M)	65/100	HIGH	French / Mistral arch.

All four models independently classify the file as HIGH opacity — a convergent finding across two training lineages and three model sizes. The 18-point spread between Apertus 8B and Mistral 7B is within the acceptable inter-model variance range (the methodology flags spreads exceeding 30 points for calibration review). The convergence on classification despite this spread is itself a finding: the HIGH designation is robust to both model size and training lineage.

A.2 Dimension Convergence

The dimension-by-dimension comparison reveals a consistent structural profile across all four models:

DIMENSION	APERTUS 8B	MISTRAL 7B	APERTUS 70B	MISTRAL NEMO	SIGNAL
Undeclared preconditions	12	12	12	12	✓ 4× convergent
Global state risk	16	8	14	15	✓ convergent
Implicit coupling	14	10	15	10	✓ convergent
Contract completeness	12	12	10	10	✓ convergent
Structural centrality	18	18	18	18	✓ 4× convergent

`Structural centrality` scores 18/20 across all four models — a 4× confirmed finding. `Undeclared preconditions` scores 12/20 across all four models — a second 4× confirmed finding. These two dimensions represent the highest-confidence structural characterization of this file: it is extraordinarily central to system correctness and carries a significant load of undeclared preconditions that callers must satisfy without being told.

DEMONSTRATION CODE

The ensemble divergence methodology is demonstrated in a working Colab notebook — running Apertus and Mistral models against the LangGraph corpus and the Settele (2022) methods description analyzed in this appendix.

github.com/ActionPace/ensemble-divergence

A.3 Load-Bearing Functions — 3× Confirmed

Six functions are named as load-bearing by three or more of the four models:

```
3× confirmed (Apertus 8B, Apertus 70B, Mistral 7B or Nemo):
  → get_subgraphs
  → aget_subgraphs
  → get_input_schema
  → get_input_jsonschema
  → get_output_schema
  → get_output_jsonschema

1× (Mistral 7B and Nemo only):
  → get_config_jsonschema
  → get_context_jsonschema
```

In the paper's methodology, functions named by three independent models are treated as confirmed composition seams — the highest-confidence attachment points for a governance wrapper layer. The six 3× confirmed functions represent the schema and subgraph interface layer of the Pregel implementation. The deprecated `get_config_jsonschema` function named by the Mistral models but not the Apertus models is a divergence zone: a zone where the two training lineages locate different structural layers of the same file.

A.4 Primary Signal — The 70B Depth-of-Read Finding

The most precise primary signal comes from Apertus 70B, which names a specific undeclared contract not visible to the smaller models:

```
Apertus 8B:
  "This file has high opacity due to its reliance on global state,
  implicit coupling, and incomplete contracts."

Mistral 7B:
  "This file has a moderate level of opacity, with assumptions about
  other parts of the system, global state usage, and incomplete contracts."

Apertus 70B:
  "The get_subgraphs method assumes that subgraphs are stored in a nested
  structure with namespaces separated by NS_SEP, which is not declared in
  the type hints."

Mistral Nemo 12B:
  "High opacity due to global state usage and implicit coupling."
```

The 70B model goes deeper than all three smaller models and names a specific, falsifiable contract: the `NS_SEP` namespace separator convention that callers of `get_subgraphs` must satisfy without any interface declaration. This is a concrete orphaned invariant — a real structural obligation that the code depends on but no component owns. The 8B-class models (Apertus 8B, Mistral 7B, Mistral Nemo 12B) all characterize opacity at the level of general patterns; the 70B model reads past the surface interface to find the specific undeclared commitment.

A.5 Caller Burden — The Depth-of-Read Divergence

The caller burden analysis reproduces and strengthens the Appendix A finding from the original two-model run. Three models read the interface as sufficient; one model finds unstated requirements:

```
Apertus 8B:   UNSTATED items found
  [X] The exact input and output schemas for the graph's nodes and edges

Mistral 7B:   ALL STATED

Apertus 70B:  UNSTATED items found
  [X] get_subgraphs and aget_subgraphs require a namespace to filter
      subgraphs by; recurse determines recursive traversal behavior
  [X] NS_SEP convention for nested subgraph namespace structure

Mistral Nemo 12B:  ALL STATED
```

The pattern is consistent: the two Mistral models read the high-level interface and conclude that the caller burden is stated. The two Apertus models — at both 8B and 70B scale — find unstated requirements. This is not a size effect; it is an architectural effect. The Apertus models, trained differently from Mistral, locate a different layer of structural obligation in the same file. The 70B model finds the most specific obligation: the `NS_SEP` convention and namespace filtering requirement that callers must know to use `get_subgraphs` correctly.

This is the methodological point at full scale. Agreement across training lineages establishes high-confidence structural findings — the six 3× load-bearing functions and the 4× dimension convergence on structural centrality and undeclared preconditions. Disagreement across training lineages locates composition seams — the zones where the boundary between this file's responsibility and its callers' is not clearly drawn, and where different training histories lead independent models to different structural interpretations. The most valuable finding is neither the agreement nor the disagreement alone, but the pattern of where each occurs.

APPENDIX A · NOTE ON METHODOLOGY EVOLUTION

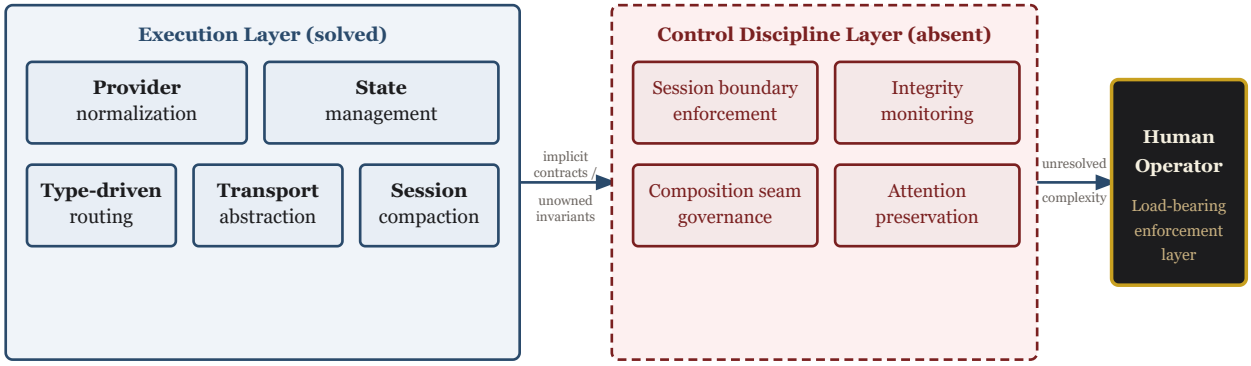
From Two-Model to Four-Model Ensemble

The original survey used a three-model ensemble of Claude Haiku, Gemini Flash, and Gemini 2.5 Flash Lite across 1,269 files. This worked example uses a four-model ensemble of Apertus 8B, Mistral 7B, Apertus 70B, and Mistral Nemo 12B against a single file from the LangGraph survey corpus. The two configurations are not directly comparable — the original survey used API-based models optimized for the map phase, while this demonstration uses locally-run GGUF models optimized for public reproducibility without proprietary API access.

What the demonstration establishes is that the structural signals identified in the original survey are robust to model substitution. The six 3× confirmed load-bearing functions, the 4× convergent dimension scores on structural centrality and undeclared preconditions, and the depth-of-read divergence pattern are all consistent with the findings documented in the paper. The methodology generalizes across model families and deployment configurations.

DIAGRAM 1

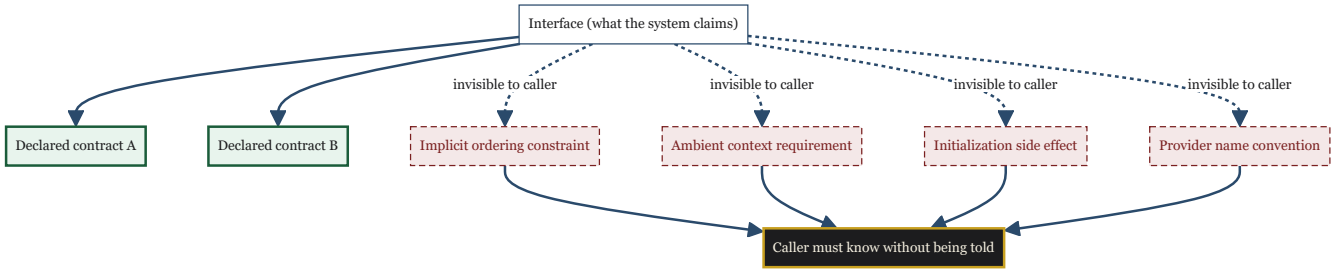
The Execution–Control Imbalance



When the control discipline layer is absent, implicit contracts accumulate and transfer to the human operator. The practitioner becomes the enforcement layer for what the system cannot govern from within its own architectural commitments.

DIAGRAM 2

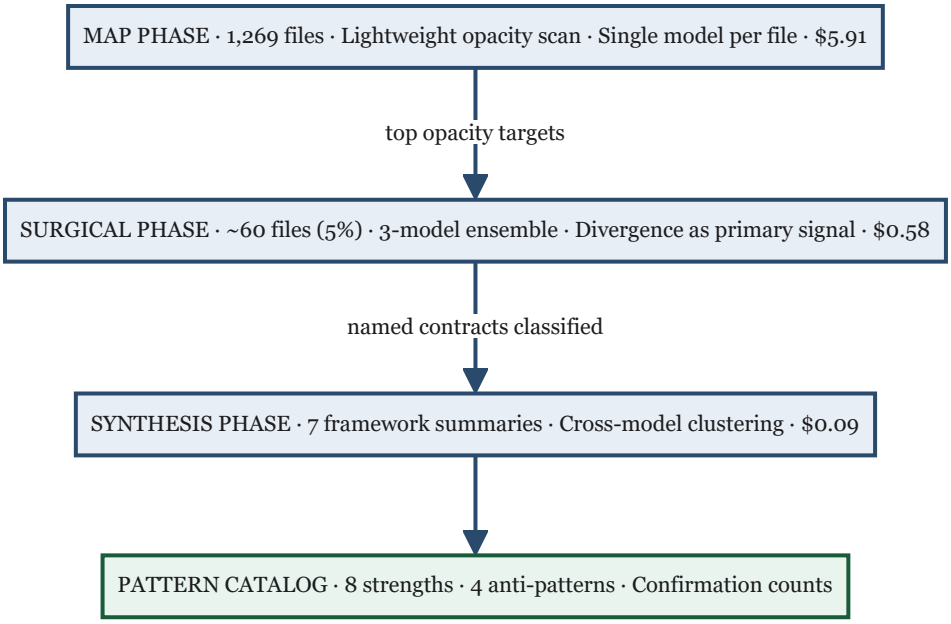
The Contract Accumulation Pattern



Orphaned invariants are invisible at the interface boundary. They accumulate silently and become apparent only when violated. The caller must satisfy them without being told they exist.

DIAGRAM 3

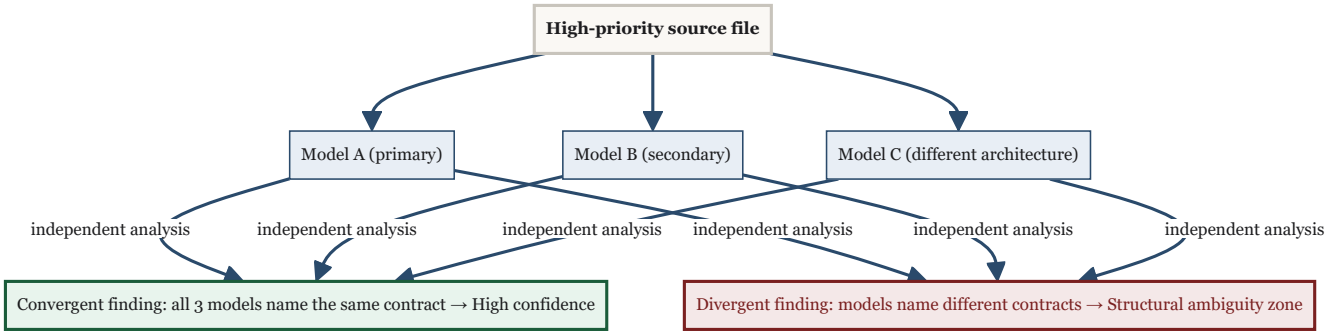
Three-Phase Survey Funnel



The three-phase funnel concentrates analytical cost on the files most likely to yield structural findings. The 5% surgical selection accounts for 8.8% of total cost while producing 100% of the confirmed pattern findings. Total analysis cost: $6.58.

DIAGRAM 4

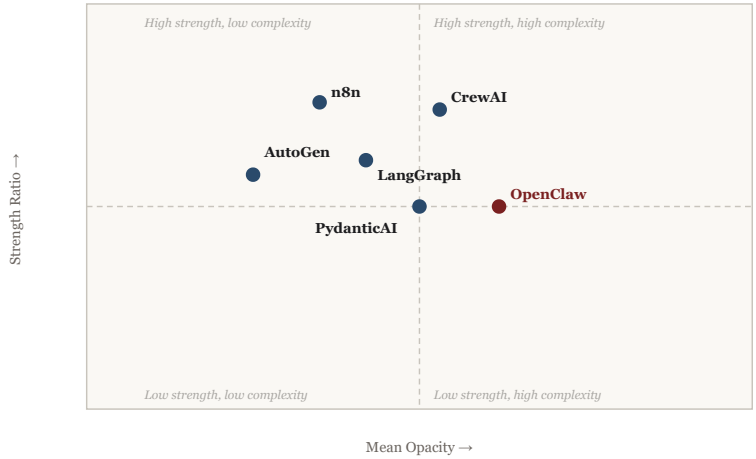
Ensemble Divergence as Primary Signal



Disagreement between models identifies zones where structural interpretation is genuinely ambiguous. Convergence across architecturally different models indicates findings robust to model-specific training biases.

DIAGRAM 5

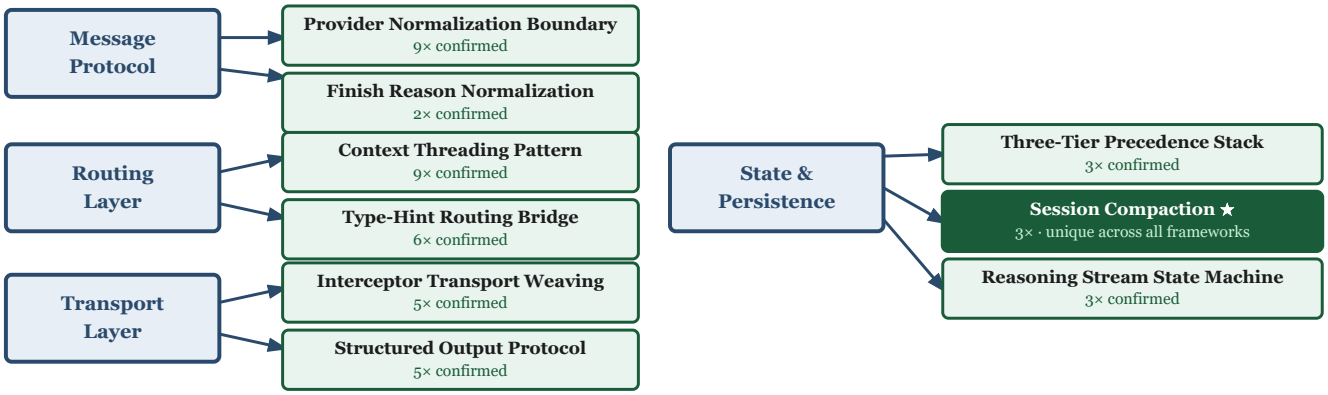
Strength Ratio vs. Mean Opacity — Six Systems



AutoGen is the cleanest codebase with moderate strength ratio. n8n and CrewAI score highest on strength despite different opacity profiles. OpenClaw carries the highest complexity, reflecting its ambition to solve problems the other systems do not attempt.

DIAGRAM 6

Pattern Catalog — Strengths by Architectural Layer

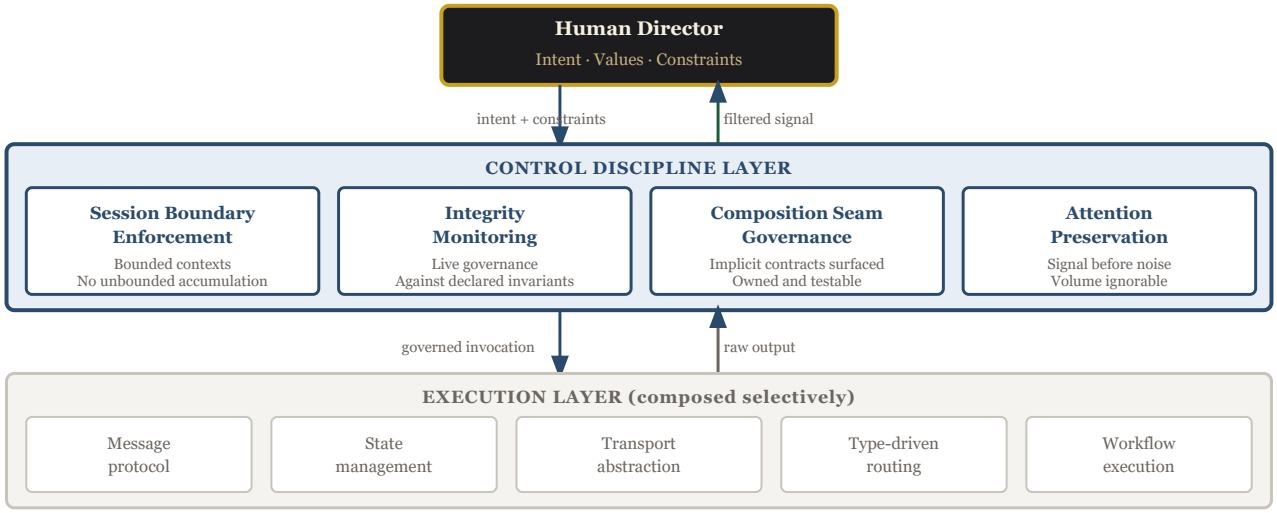


Eight strengths across four architectural layers, each independently confirmed across multiple frameworks.

★ Session Compaction Lifecycle — the only pattern in the catalog with no equivalent in any other surveyed system. Represents an architectural commitment to long-running session context management that other frameworks have not attempted.

DIAGRAM 7

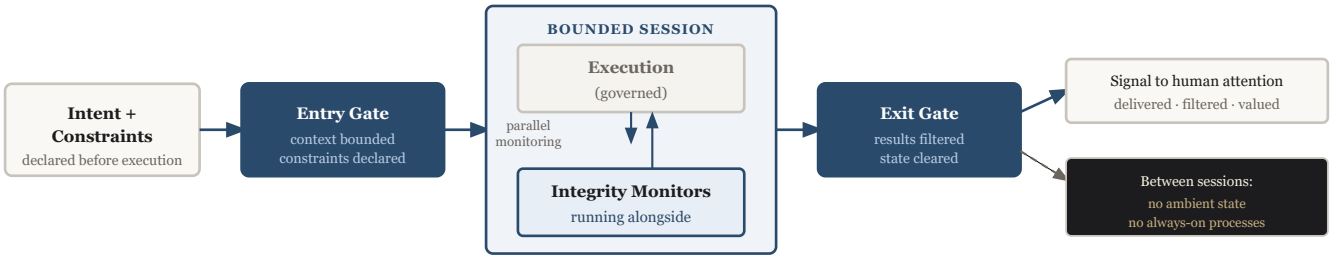
The Control Discipline Layer — Four Commitments



The control discipline layer sits between the human director and the execution layer. It governs when capabilities are invoked, monitors outputs against declared invariants, enforces session boundaries, and filters what reaches human attention.

DIAGRAM 8

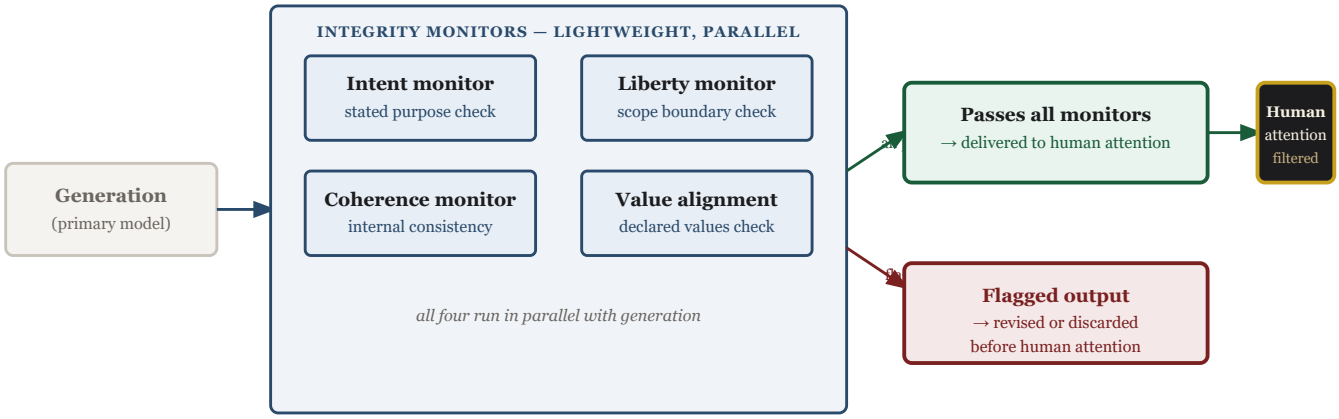
Session Boundary Enforcement — The Constitutional Model



Session boundary enforcement is constitutional, not censorial. Rules are established before execution begins, not applied as interruptions during it. Between sessions, the system holds no ambient state.

DIAGRAM 9

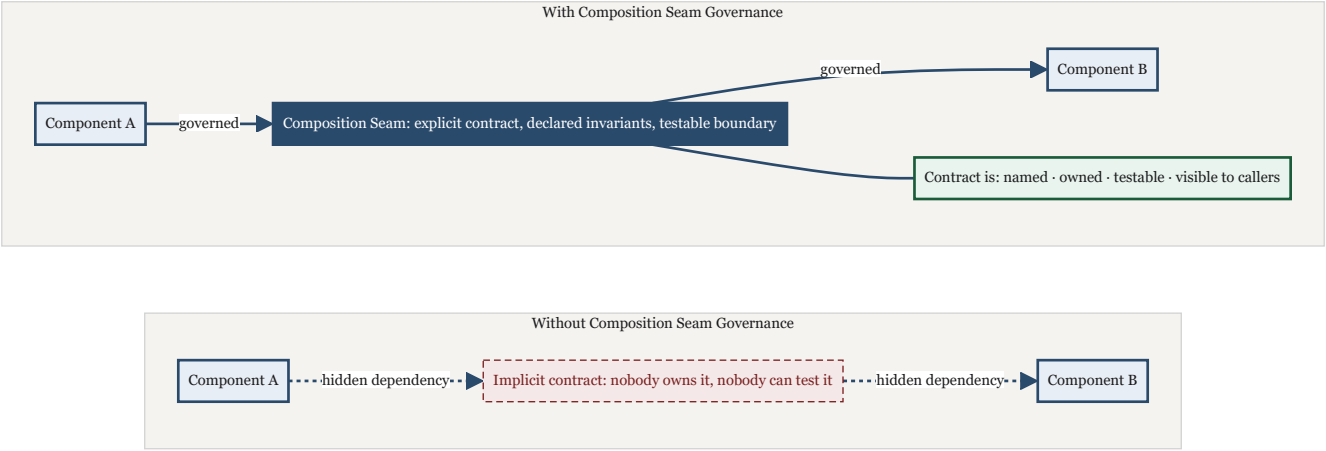
Integrity Monitoring — Parallel Governance Structure



Integrity monitors run in parallel with generation, not after it. Their function is to filter what reaches human attention — separating signal from volume before the practitioner must evaluate it.

DIAGRAM 10

Composition Seam Governance — Making Implicit Contracts Explicit



Composition seam governance does not add new contracts — it makes existing implicit contracts explicit, gives them owners, and makes them testable. The orphaned invariant that previously lived only in the space between components now has a home.